Stored XSS into Onclick Event with Angle Brackets and Double Quotes HTML-Encoded and Single Quotes and Backslash Escaped

1. Overview

Field	Details
Vulnerability	Stored Cross-Site Scripting (XSS)
Category	Client-Side Injection
Injection Context	HTML Event Handler Attribute
Affected Attribute	`onclick`
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy
Vulnerability Type	Stored XSS

2. Description

Stored Cross-Site Scripting (XSS) occurs when an application stores user-controlled input and later displays it to other users without applying appropriate output encoding.

In this lab, the application stores user input that is subsequently reflected inside an HTML `onclick` event-handler attribute.

Although the application encodes angle brackets and double quotes and escapes single quotes and backslashes, these protections may be insufficient when the input is interpreted within a JavaScript execution context.

An attacker may exploit the incorrect handling of JavaScript syntax to inject executable code into the event handler.

3. Root Cause

The root cause is insufficient context-aware output encoding when handling untrusted data inside JavaScript event-handler attributes.

HTML attribute encoding and JavaScript string encoding address different parsing contexts. Applying one encoding mechanism without considering the complete output context can leave an injection vulnerability.

The use of inline event handlers further increases the risk of JavaScript injection.

4. Proof of Concept (PoC)

Testing environment: Authorized PortSwigger Web Security Academy lab.

1. Open the assigned lab.
2. Identify a functionality that stores user-controlled input, such as a comment or other submitted content.
3. Submit a harmless test value.
4. Inspect the rendered HTML and locate the corresponding `onclick` attribute.
5. Examine how the application handles HTML-sensitive characters and JavaScript string delimiters.
6. Determine whether the JavaScript string context can be escaped despite the implemented filtering.
7. Verify execution using a harmless proof-of-concept in the authorized lab.

Example context to inspect:

```
None
<button onclick="handleClick('USER_INPUT')">
    Click me
</button>
```

The example illustrates the vulnerable output context, not necessarily the exact lab implementation or a working payload.

5. Security Impact

Successful exploitation may result in:

- Arbitrary JavaScript execution in the application's origin.
- Persistent execution when other users view the affected content.

- Unauthorized actions through a victim's authenticated session.
- Manipulation of page content and user interactions.
- Exposure of information accessible to client-side scripts.
- Phishing attacks using trusted application content.

Stored XSS can have a wider impact because malicious input remains stored and may affect multiple users.

6. Recommended Remediation

- Avoid inserting untrusted data into inline JavaScript event handlers.
- Replace inline event handlers with JavaScript event listeners.
- Use safe DOM APIs and secure templating frameworks.
- Apply context-specific encoding based on the final output location.
- Treat stored user input as untrusted whenever it is rendered.
- Avoid relying on blacklist-based character filtering.
- Implement a restrictive Content Security Policy (CSP).
- Add automated regression tests for HTML and JavaScript injection contexts.

7. Verification

After remediation:

1. Submit the original test input again.
2. Inspect the rendered HTML.
3. Confirm that user input cannot modify the JavaScript execution context.
4. Verify that stored content is displayed safely.
5. Test the functionality using different special-character combinations.

8. References

- PortSwigger: Stored XSS
- PortSwigger: XSS Contexts
- OWASP: Cross Site Scripting Prevention Cheat Sheet

9. Disclaimer

This report is intended for educational purposes and authorized security testing only. Testing should only be performed against systems for which explicit permission has been granted.